

检查点 4 实验报告

极简 Backbone 搭建、极简 YOLO Loss 与训练跑通

2026 年 4 月 18 日

目录

1 实验目标	3
2 工程结构	3
3 模型设计	3
3.1 极简 Backbone	3
3.2 Detection Head	4
3.3 张量形状说明	4
4 标签编码方式	5
4.1 为什么使用网格坐标	5
4.2 标签张量设计	6
4.3 标签编码过程	6
5 损失函数设计	7
5.1 损失组成	7
5.2 损失计算方式	7
6 数据与训练设置	8
6.1 数据来源	8
6.2 训练超参数	8
7 训练结果与曲线分析	8
7.1 训练结果	8
7.2 Loss 曲线	9
7.3 曲线解释	10
8 预测框与真实框可视化	11
8.1 可视化方式	11
8.2 可视化结果	12

8.3 结果解释	13
9 结论	14
A 附录：脚本与工具作用说明	14

1 实验目标

本次检查点 4 的目标是完成一个最小可训练的目标检测链路，重点不在检测精度，而在于把模型、标签编码、损失函数和训练流程真正跑通。具体目标包括：

1. 手写一个 10-20 层的极简 CNN backbone。
2. 手写一个简化版 YOLOv1 风格损失函数，只保留格子分类与 bbox 中心点、宽高回归。
3. 跑通前向、反向训练流程，并使用 TensorBoard 记录 training loss 曲线。
4. 在真实样本上可视化当前模型的预测框与真实框对比。

2 工程结构

本次实验在 `/home/lidz/YOLO/miniyolo` 下按真实工程项目方式组织代码，主要目录如下：

- `models/`: 模型结构，包含 backbone、detection head 和完整 MiniYOLO。
- `data/`: 真实数据读取与标签编码。
- `losses/`: 极简 YOLO 损失函数。
- `tools/`: 模型检查、训练、测试、可视化与曲线导出脚本。
- `runs/`: TensorBoard 日志目录。
- `outputs/`: 模型权重、预测可视化结果与训练曲线图片。
- `logs/`: 材料汇总与实验记录。
- `report/`: LaTeX 报告与最终 PDF。

3 模型设计

3.1 极简 Backbone

本次实验设计的 backbone 仅使用最基础的 Conv2d + ReLU + MaxPool2d 组合。输入图像大小统一为 224x224，通过 5 次 2x2 池化，将空间分辨率逐步压缩为 7x7。最终输出特征图大小为：

$$[B, 512, 7, 7]$$

之所以选择 224 作为输入尺寸，是因为经过连续 5 次池化后可以稳定得到 224 -> 112 -> 56 -> 28 -> 14 -> 7，便于后续采用 YOLOv1 风格的 7x7 网格预测。

3.2 Detection Head

在 backbone 输出特征图 $[B, 512, 7, 7]$ 的基础上, 进一步接一个极简 detection head:

- 先通过一个 3×3 卷积将通道从 512 压缩到 256
- 再通过一个 1×1 卷积得到最终预测维度

本实验设定类别数为 20, 每个格子同时预测一个 bbox 参数向量 (x, y, w, h) , 因此每个格子的输出维度为:

$$20 + 4 = 24$$

所以 detection head 最终输出为:

$$[B, 7, 7, 24]$$

其中:

- 前 20 维表示分类预测
- 后 4 维表示 bbox 回归值

3.3 张量形状说明

利用 `tools/inspect_model.py` 对 backbone 与 head 的张量形状进行了逐层检查, 结果如表 1 所示。

表 1: 极简 backbone 的逐层 tensor shape

层编号	模块类型	输出张量形状
input	输入图像	[2, 3, 224, 224]
layer0	ConvBlock	[2, 16, 224, 224]
layer1	ConvBlock	[2, 16, 224, 224]
layer2	MaxPool2d	[2, 16, 112, 112]
layer3	ConvBlock	[2, 32, 112, 112]
layer4	ConvBlock	[2, 32, 112, 112]
layer5	MaxPool2d	[2, 32, 56, 56]
layer6	ConvBlock	[2, 64, 56, 56]
layer7	ConvBlock	[2, 64, 56, 56]
layer8	MaxPool2d	[2, 64, 28, 28]
layer9	ConvBlock	[2, 128, 28, 28]
layer10	ConvBlock	[2, 128, 28, 28]
layer11	MaxPool2d	[2, 128, 14, 14]
layer12	ConvBlock	[2, 256, 14, 14]
layer13	ConvBlock	[2, 256, 14, 14]
layer14	MaxPool2d	[2, 256, 7, 7]
layer15	ConvBlock	[2, 512, 7, 7]
layer16	ConvBlock	[2, 512, 7, 7]
output	backbone 输出	[2, 512, 7, 7]
head output	最终预测	[2, 7, 7, 24]

进一步将检测输出拆分后，可得到：

- 分类预测：class_pred = [B, 7, 7, 20]
- 框回归预测：bbox_pred = [B, 7, 7, 4]

4 标签编码方式

4.1 为什么使用网格坐标

本实验采用 YOLOv1 风格的网格监督方式。对于每个真实框，不直接用单一的 x , y 监督，而是拆成两部分：

1. grid_x, grid_y: 表示目标中心点落在哪个网格单元
2. cell_x, cell_y: 表示目标中心点在当前网格内部的相对偏移

这样可以明确回答两个问题：

- 哪个格子负责当前目标
- 目标中心点在该格子内部的精确位置

4.2 标签张量设计

为了与模型预测 [B, 7, 7, 24] 对齐, 本实验将监督信息拆成 3 个张量:

- target_cls: [7, 7, 20]
- target_box: [7, 7, 4]
- object_mask: [7, 7]

其含义如下:

- target_cls: 当前格子的分类 one-hot 监督
- target_box: 当前格子的 bbox 回归监督, 内容为 (cell_x, cell_y, w, h)
- object_mask: 该格子是否负责一个目标, 取值为 0 或 1

4.3 标签编码过程

真实框原始格式采用 xyxy, 即:

$$[x_1, y_1, x_2, y_2]$$

首先转换为:

$$c_x = \frac{x_1 + x_2}{2}, \quad c_y = \frac{y_1 + y_2}{2}, \quad w = x_2 - x_1, \quad h = y_2 - y_1$$

然后对 c_x, c_y, w, h 做图像大小归一化, 再通过 grid_size=7 映射到目标格子:

$$grid_x = \lfloor c_x \times 7 \rfloor, \quad grid_y = \lfloor c_y \times 7 \rfloor$$

格子内部偏移为:

$$cell_x = c_x \times 7 - grid_x, \quad cell_y = c_y \times 7 - grid_y$$

最后将:

- object_mask[grid_y, grid_x] = 1
- target_cls[grid_y, grid_x, label] = 1
- target_box[grid_y, grid_x] = [cell_x, cell_y, w, h]

5 损失函数设计

5.1 损失组成

本实验的极简 YOLO Loss 只保留最核心的两部分：

1. 格子分类损失
2. bbox 回归损失

不包含：

- 多尺度预测
- anchor
- objectness / confidence 分支

5.2 损失计算方式

记模型输出为：

$$pred \in \mathbb{R}^{B \times 7 \times 7 \times 24}$$

将其拆成：

$$pred_cls \in \mathbb{R}^{B \times 7 \times 7 \times 20}, \quad pred_box \in \mathbb{R}^{B \times 7 \times 7 \times 4}$$

利用 `object_mask` 只在正样本网格上计算损失。分类损失和框损失都采用均方误差 (MSE)：

$$L_{cls} = \frac{\sum((pred_cls - target_cls)^2 \cdot mask)}{N_{pos} \times 20}$$
$$L_{box} = \frac{\sum((pred_box - target_box)^2 \cdot mask)}{N_{pos} \times 4}$$

总损失定义为：

$$L = \lambda_{cls} \cdot L_{cls} + \lambda_{box} \cdot L_{box}$$

其中本实验采用：

$$\lambda_{cls} = 1.0, \quad \lambda_{box} = 5.0$$

即：

$$L = L_{cls} + 5 \cdot L_{box}$$

这样设置的原因是，bbox 回归通常比分类更难收敛，因此给予更高权重。

6 数据与训练设置

6.1 数据来源

真实训练数据来自统一后的 manifest:

- `/home/lidz/YOLO/DataSet/Unified/manifests/all_val.jsonl`

本实验为快速验证链路, 使用:

- `max_samples = 16`

图片通过 `DetectionDataset` 读取, 并统一 `resize` 到:

- `224x224`

`bounding box` 也会同步按缩放比例变换。

6.2 训练超参数

本次真实训练采用如下设置:

表 2: 训练超参数设置

项目	设置
数据源	<code>all_val.jsonl</code>
最大样本数	<code>16</code>
输入尺寸	<code>224</code>
网格大小	<code>7</code>
类别数	<code>20</code>
batch size	<code>4</code>
epochs	<code>2</code>
学习率	<code>1e-3</code>
优化器	<code>Adam</code>
设备	<code>CUDA</code>
TensorBoard 日志目录	<code>runs/real_train</code>
模型权重输出	<code>outputs/miniyolo_real_last.pth</code>

7 训练结果与曲线分析

7.1 训练结果

真实数据训练输出的 epoch 级结果如下:

- epoch 1:

- total loss = 1.6907
 - cls loss = 0.0792
 - box loss = 0.3223
- epoch 2:
 - total loss = 1.2641
 - cls loss = 0.0528
 - box loss = 0.2423

可以看到：

- 总损失从 1.6907 下降到 1.2641
- 分类损失从 0.0792 下降到 0.0528
- 框回归损失从 0.3223 下降到 0.2423

说明在这组极小规模真实训练中，模型已经表现出初步收敛趋势。

7.2 Loss 曲线

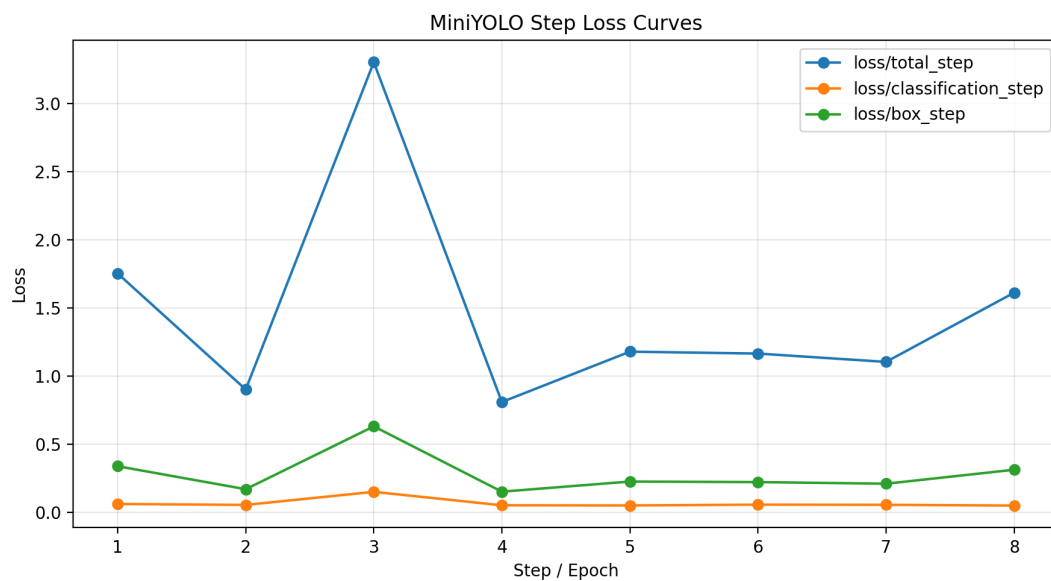


图 1: step 级别 loss 曲线

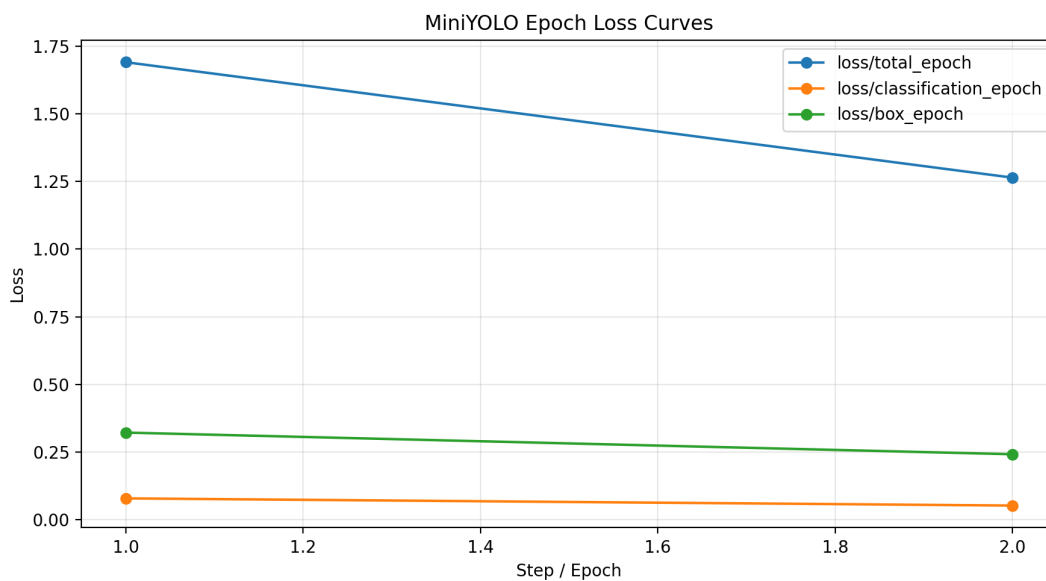


图 2: epoch 级别 loss 曲线

7.3 曲线解释

本实验导出的 loss 曲线分为两种：

1. step 曲线：

- loss/total_step
- loss/classification_step
- loss/box_step

这些值是每个 batch 直接计算得到的原始 loss 值，因此波动较大。

2. epoch 曲线：

- loss/total_epoch
- loss/classification_epoch
- loss/box_epoch

这些值是一个 epoch 内所有 batch loss 的平均值，更适合用来观察整体下降趋势。

从曲线可以看出：

- step 曲线存在明显抖动，这是因为当前训练样本数很少、batch 也很小
- epoch 曲线整体下降，说明模型已经开始学习
- 由于总损失定义为 $cls + 5 * box$ ，所以总损失的变化主要受 bbox 回归项影响

8 预测框与真实框可视化

8.1 可视化方式

在 `tools/visualize_predictions.py` 中，使用训练后权重：

- `outputs/miniyolo_real_last.pth`

对真实样本进行推理，并将预测框与真实框画在同一张图上。可视化规则如下：

- 绿色框：真实框 (ground truth)
- 绿色文字： `gt:{label}`
- 红色框：模型预测框
- 红色文字： `pred:{class_id} {score} ({gy},{gx})`

其中：

- `class_id` 表示预测类别编号
- `score` 表示该预测的分类分数
- `(gy, gx)` 表示该预测来自哪个网格位置

为了保证图中至少能看到红色预测框，本次可视化时将分数阈值设为：

- `score_threshold = 0.0`

并设置：

- `top_k = 3`

即每张图最多展示 3 个预测框。

8.2 可视化结果

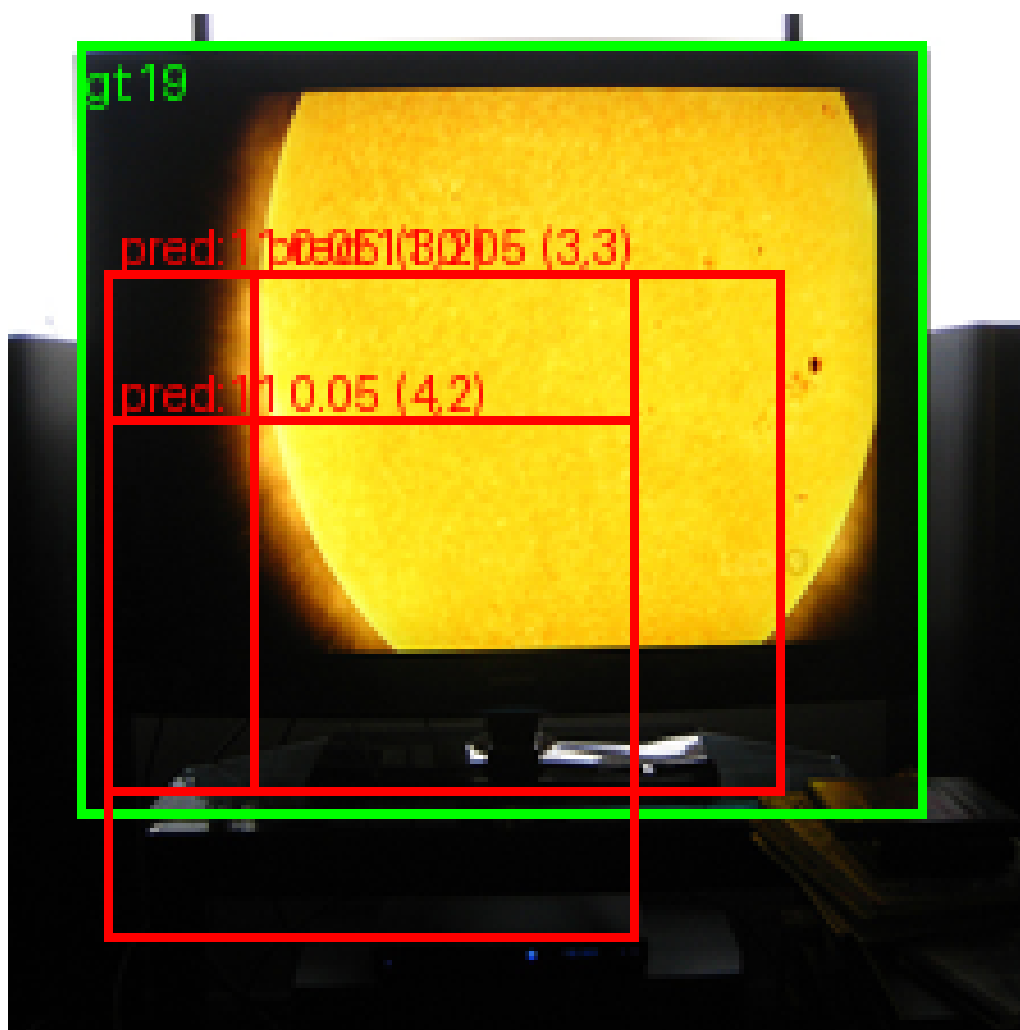


图 3: 样本 voc2012_val_2008_000002 的预测框与真实框对比

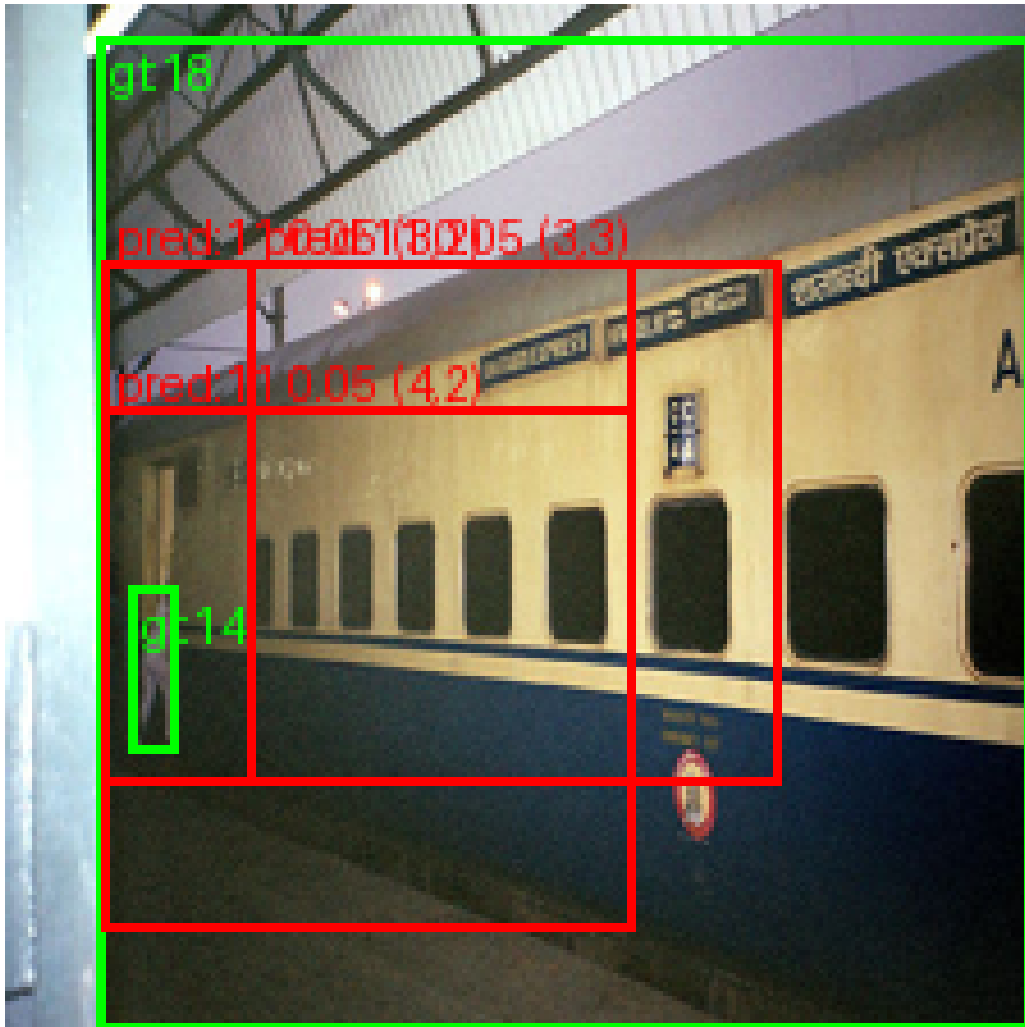


图 4: 样本 voc2012_val_2008_000003 的预测框与真实框对比

8.3 结果解释

当前极简模型在可视化结果中的红色预测框较多偏差，和真实框并不吻合，这是当前阶段的正常现象，原因主要包括：

- 训练样本数非常少，仅使用了 16 个样本
- 训练 epoch 很少，仅训练 2 个 epoch
- 模型结构极简，不包含更强的检测设计
- 当前损失函数不包含 objectness/confidence 分支

因此，本实验阶段的目标不是追求检测精度，而是验证：

- 模型可以输出预测框
- 预测结果可以成功解码并可视化
- 能与真实框放在同一张图上进行对比

9 结论

本次检查点 4 已完成一个最小可训练目标检测系统的搭建，并验证了从输入图像到检测输出、标签编码、损失计算、训练记录和结果可视化的完整链路。实验结论如下：

1. 手写的极简 backbone 能稳定将 224x224 输入压缩到 [B, 512, 7, 7] 特征图。
2. detection head 能将特征图进一步映射为 [B, 7, 7, 24] 的 YOLO 风格预测张量。
3. 通过 `target_encoder.py`, 真实框可以被正确编码为 `target_cls`, `target_box` 和 `object_mask`。
4. 极简 YOLO Loss 能输出标量 training loss，并支持反向传播。
5. 在真实数据上训练 2 个 epoch 后，epoch 平均总损失从 1.6907 下降到 1.2641，说明模型已经具备基本学习能力。
6. 当前预测框质量仍然较差，但已经成功完成与真实框的同图对比，这满足了检查点 4 对“跑通链路可视化”的核心要求。

A 附录：脚本与工具作用说明

脚本/文件	作用
<code>models/minimal_backbone.py</code>	定义极简 CNN backbone，负责把输入图像压缩为 7x7 特征图。
<code>models/detection_head.py</code>	定义 detection head，将 backbone 特征图映射为 YOLO 风格预测输出。
<code>models/miniyolo.py</code>	将 backbone 与 detection head 封装为完整的 MiniYOLO 模型。
<code>losses/minimal_yolo_loss.py</code>	定义极简 YOLO Loss，计算分类损失、bbox 回归损失和总损失。
<code>data/target_encoder.py</code>	将真实框和类别编码为 <code>target_cls</code> 、 <code>target_box</code> 和 <code>object_mask</code> 。
<code>data/detection_dataset.py</code>	从统一 manifest 读取真实样本、读取图片、缩放 bbox，并生成训练所需监督张量。
<code>tools/inspect_model.py</code>	检查 backbone 与 head 的输出 shape，并记录每层 tensor shape。
<code>tools/test_target_encoder.py</code>	验证 target encoder 是否能将单个真实框正确写入网格监督张量。
<code>tools/test_minimal_yolo_loss.py</code>	验证极简 YOLO Loss 是否能输出标量并支持 <code>backward()</code> 。
<code>tools/test_detection_dataset.py</code>	验证真实数据版 Dataset 与 DataLoader 是否工作正常。

脚本/文件	作用
<code>tools/train_minimal.py</code>	使用 toy data 构造最小训练闭环，验证模型、loss 和优化器能协同工作。
<code>tools/train_with_tensorboard.py</code>	使用 toy data 训练并写出 TensorBoard loss 日志。
<code>tools/train_real.py</code>	使用真实统一数据进行训练，记录 loss 曲线并保存模型权重。
<code>tools/visualize_predictions.py</code>	加载训练后权重，在真实样本上画出预测框与真实框对比图。
<code>tools/export_training_curves.py</code>	从 TensorBoard 事件文件中导出 step/epoch loss 曲线图片。
<code>TensorBoard</code>	浏览训练过程中的 loss 变化曲线。
<code>runs/</code>	保存 TensorBoard 事件文件。
<code>outputs/</code>	保存训练权重、预测可视化图与 loss 曲线导出图片。
<code>logs/checkpoint4_materials.md</code>	汇总 checkpoint 4 当前可直接用于报告的材料。